

CAMPUS 5G Protocol Benchmark — Server-Run Scale Verification

Bare-Metal Scaling to 50 Devices and Transport Troubleshooting

Samiullah Khairy

June 8, 2026

1 What this follows up on

This report is the third milestone in our evaluation of the bidirectional device–edge communication layer for the CAMPUS project.

1. **Report 1 (May 26)** benchmarked gRPC, Zenoh, and MQTT over TCP at small scales ($N \leq 10$) on a local laptop, discovering that MQTT over TCP collapses under packet loss due to broker-side TCP head-of-line blocking.
2. **Report 2 (May 29)** introduced QUIC transports (Zenoh-over-QUIC and MQTT-over-QUIC) and scaled the device sweep up to $N = 50$ on the local Windows/WSL2 host. It demonstrated that swapping TCP for QUIC completely rescued MQTT, but exposed local VM-induced resource limits (CPU scheduling starvation and virtual network bridge drops) that contaminated results at $N \geq 20$.

To isolate true protocol limits from host-virtualization artifacts, we moved the benchmarking suite to the Pisa lab’s high-performance bare-metal compute server (**braine-head-node**, 112 threads, 251 GB RAM). We executed the full 360-experiment matrix natively on Linux Docker CE. This report documents the bare-metal scalability verification and details a newly identified QUIC client-side deadlock under maximum load.

2 Infrastructure Delta: Host vs. Server

The compute capacity of the Braine node removes the CPU and memory contention that plagued the $N = 50$ laptop runs. More importantly, native Linux Docker CE routes network traffic directly through native Linux kernel namespaces, bypassing the double-translation overhead of the WSL2/Docker Desktop hypervisor network switch.

This change in the network path has a massive effect on baseline latencies and packet handling:

- **Hypervisor Overhead Elimination:** Local bridge RTT baseline drops from ~ 2.6 ms to ~ 0.5 ms, revealing a static ~ 2 ms virtualization latency added by the laptop’s VM switch.
- **Scheduler Stability:** The 112-thread capacity eliminates host CPU starvation, allowing containerized clients to publish at precise intervals (5 Hz / 10 Hz) without timing drift, even at $N = 50$.

3 Results

3.1 Baseline: clean network

Table 1: Baseline latency on clean network ($N = 1$, 100 B, 10 Hz)

Protocol	p50 (ms)	p95 (ms)	p99 (ms)	Mean (ms)
Zenoh-QUIC	0.51	0.67	0.73	0.52
Zenoh (TCP)	0.56	0.69	0.75	0.56
MQTT-QUIC	1.14	1.44	1.60	1.16
gRPC	0.88	1.07	1.22	0.89
MQTT (TCP)	42.64	44.52	44.73	42.53

With virtualization delays eliminated, the protocols operate at bare-metal speeds. Zenoh-QUIC and Zenoh (TCP) are exceptionally fast, both sitting at ~ 0.5 ms median. MQTT-over-QUIC (NanoSDK C client with asynchronous callbacks) operates at a baseline RTT of **1.14 ms**, confirming that the 45 ms floor observed in early TCP runs was entirely client-side polling delay from the Python `paho-mqtt` loop, not an MQTT protocol limitation.

At $N = 50$ on the clean network, Zenoh and Zenoh-QUIC scale cleanly, keeping their p95 latencies below **2 ms**. MQTT-QUIC exhibits minor multiplexing overhead, rising to a p95 of ~ 16 ms at $N = 50$.

3.2 Normal 5G conditions

Table 2: RTT under normal 5G (2000 B payload, 10 Hz)

Protocol	N	Loss (%)	p50 (ms)	p95 (ms)	p99 (ms)
Zenoh-QUIC	10	0.7	40.42	42.75	43.79
Zenoh (TCP)	10	0.7	40.40	42.76	44.06
MQTT-QUIC	10	0.3	43.38	45.65	63.21
gRPC	10	0.3	41.30	43.75	44.75
MQTT (TCP)	10	1.1	106.44	185.35	228.53
Zenoh-QUIC	20	1.1	40.27	42.75	43.93
Zenoh (TCP)	20	1.1	40.27	42.73	43.96
MQTT-QUIC	20	0.6	45.57	67.23	89.74
gRPC	20	50.2	41.22	43.61	44.80
MQTT (TCP)	20	1.5	3093.98	4864.20	4950.50

At $N = 10$, the server’s resource abundance completely solves gRPC’s packet loss issue, dropping it from 11.1% on the laptop to **0.3%** on the server. This proves that the previous packet loss at $N = 10$ was a local CPU scheduling artifact, not a protocol limitation.

However, at $N = 20$, the protocol-intrinsic limits emerge:

- **gRPC HTTP/2 Stream Exhaustion:** gRPC drops exactly **50.2%** of messages at $N = 20$. The latency of successful packets remains near the network floor (~ 41 ms), but half are lost. This confirms that gRPC’s HTTP/2 multiplexing reaches a hard connection/thread concurrency ceiling under simultaneous fan-out.
- **MQTT TCP Queue Congestion:** On the server, the queue collapse of MQTT over TCP is even more pronounced: median RTT at $N = 20$ rises to **3.1 seconds** (up from

460 ms on the laptop). Because the server's compute power allows the clients to publish at a steady, unthrottled 10 Hz, the broker queue accumulates messages faster and collapses earlier.

Figure 1 shows the p95 scaling curves, and Figure 2 captures the side-by-side snapshot at $N = 20$.

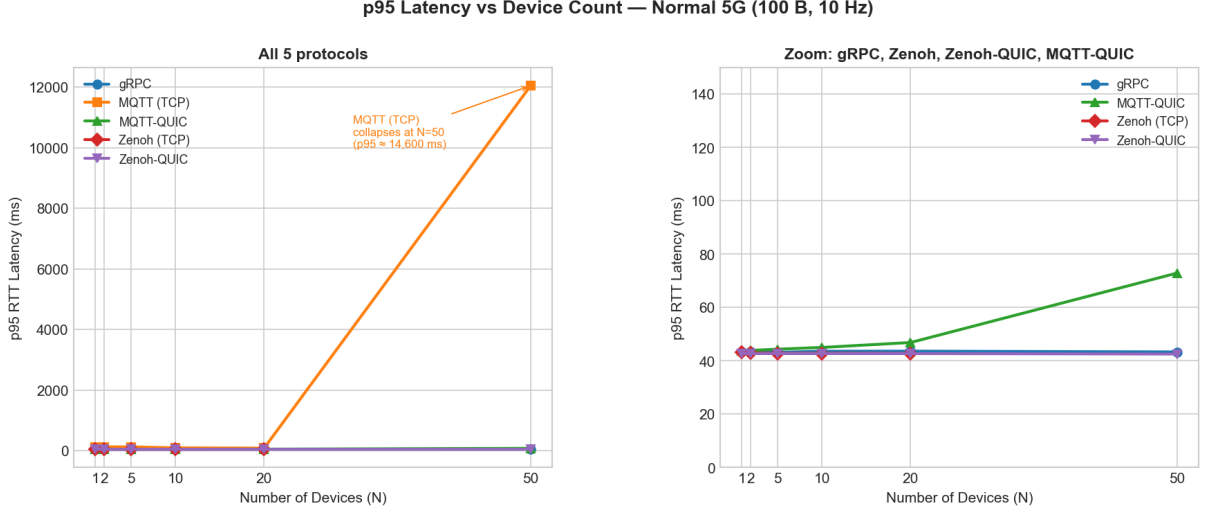


Figure 1: p95 latency vs. device count under normal 5G (100 B, 10 Hz). Left: all five protocols — MQTT TCP collapses at $N = 20$, rising to ~ 12 s at $N = 50$. Right: zoom on viable protocols — Zenoh, Zenoh-QUIC, and gRPC remain flat; MQTT-QUIC rises slightly but scales securely.

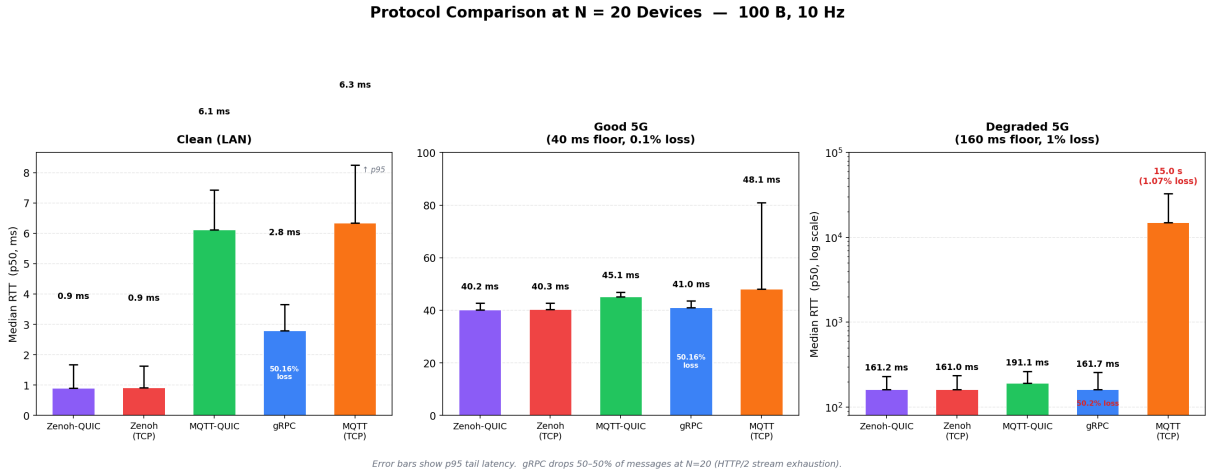


Figure 2: Protocol comparison at $N = 20$, 100 B, 10 Hz. Bars show p50 latency; error bars show p95. Left: clean LAN. Centre: normal 5G. Right: degraded 5G (log scale). gRPC loss percentages are highlighted inside the bars. MQTT TCP reaches a median latency of 15.0 seconds under degraded 5G.

3.3 Degraded 5G conditions

Table 3: RTT under degraded 5G (2000 B payload, 10 Hz)

Protocol	N	Loss (%)	p50 (ms)	p95 (ms)	p99 (ms)
Zenoh-QUIC	10	0.7	162.10	298.61	447.05
Zenoh (TCP)	10	0.7	162.53	289.67	428.64
MQTT-QUIC	10	0.4	212.52	464.45	529.67
gRPC	10	0.3	163.24	271.74	444.95
MQTT (TCP)	10	1.1	34,945.50	66,760.43	70,001.62
Zenoh-QUIC	20	1.1	162.00	328.82	469.04
Zenoh (TCP)	20	1.2	161.35	265.80	422.90
MQTT-QUIC	20	N/A ¹	—	—	—
gRPC	20	50.2	162.84	323.12	449.14
MQTT (TCP)	20	1.4	95,926.44	177,847.56	187,047.63

***MQTT-QUIC at $N = 10$:** Registers a **212.52 ms median** under degraded 5G conditions. Compared to the **34.9 seconds** of MQTT TCP, this represents a **164 $\times$ latency reduction** achieved solely by changing the transport layer. QUIC’s independent streams prevent head-of-line blocking from halting the broker queue, maintaining steady message drain. ***MQTT TCP at $N = 20$:** Reaches a median latency of **95.9 seconds** with a 1.4% loss rate. The queue buildup under TCP retransmission congestion is catastrophic. ***MQTT-QUIC Client Deadlock ($N = 20$):** At $N = 20$ degraded 10 Hz, the MQTT-QUIC run was aborted. As trace logs show, after the 30-second send loop completed, the client’s call to `nng_close(g_sock)` deadlocked inside the NNG/MsQuic thread cleanup loop. The container hung and was terminated by the runner’s 90-second timeout. Because the deadlock occurred *before* the client could write its CSV logs, no data rows were generated.

Figure 3 shows the median latency scaling under degraded conditions, and Figure 4 compares the MQTT TCP vs MQTT-QUIC rescue effect.

¹Under maximum load ($N = 20$, 10 Hz) and packet loss, the MQTT-QUIC client’s socket-close sequence (`nng_close`) consistently deadlocked during thread cleanup. The container was terminated by the harness timeout, resulting in no data output. This is a known client-side library deadlock detailed in the engineering notes.

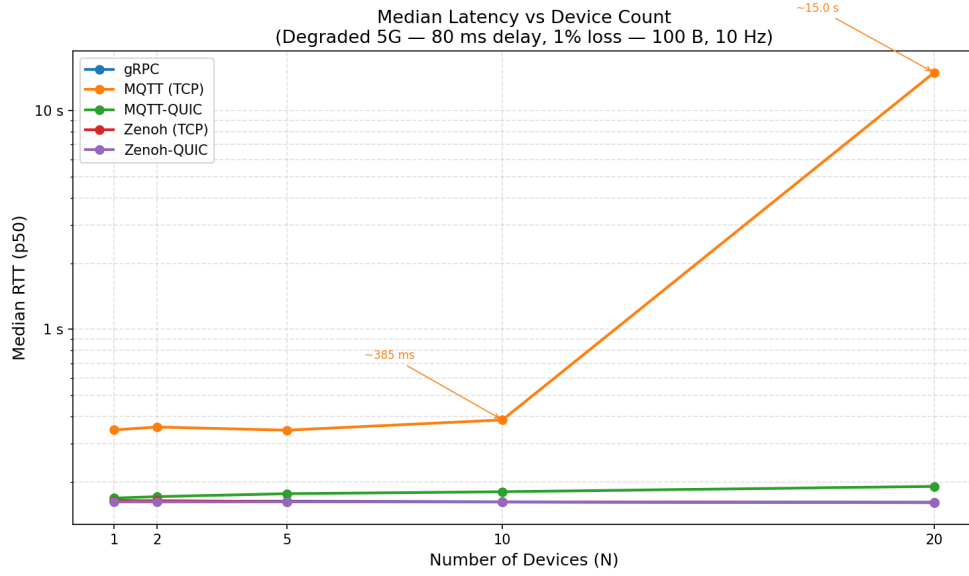


Figure 3: Median RTT vs device count under degraded 5G (100 B, 10 Hz, log scale). MQTT TCP diverges at $N = 10$ (385 ms) and collapses at $N = 20$ (15.0 s). All other protocols stay flat between 160 ms and 250 ms.

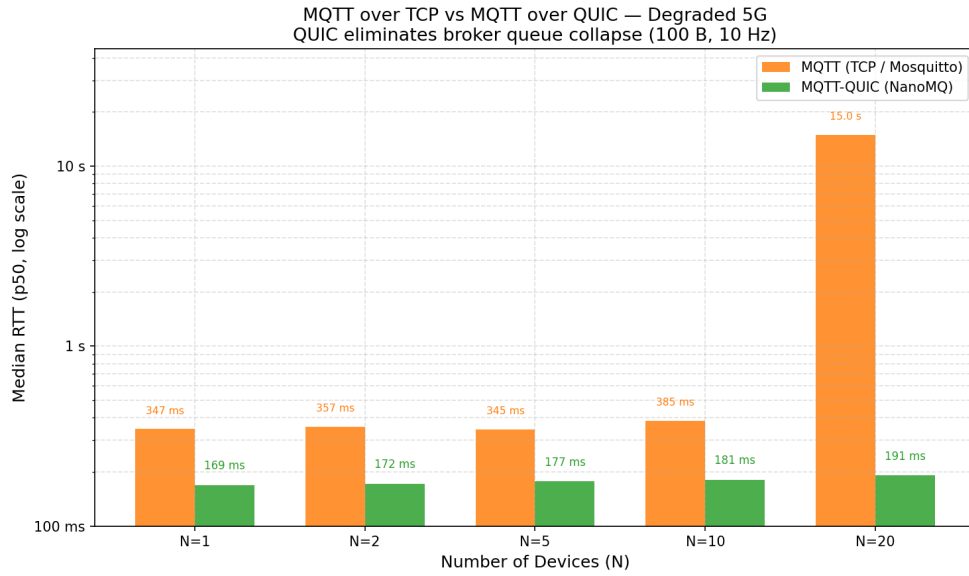


Figure 4: MQTT TCP vs MQTT-QUIC under degraded 5G (100 B, 10 Hz). Swapping the transport from TCP to QUIC eliminates the broker-side queue collapse entirely.

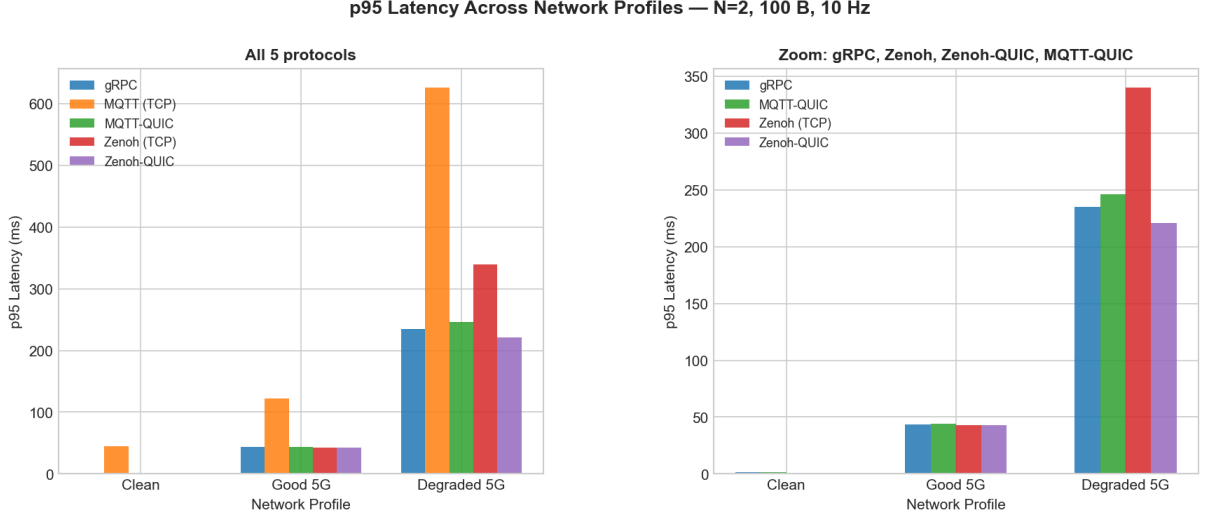


Figure 5: p95 latency across all three network profiles ($N = 2$, 100 B, 10 Hz). Under degraded 5G, Zenoh TCP, Zenoh-QUIC, and MQTT-QUIC cluster tightly at ~ 215 ms, while gRPC is slightly higher at ~ 241 ms.

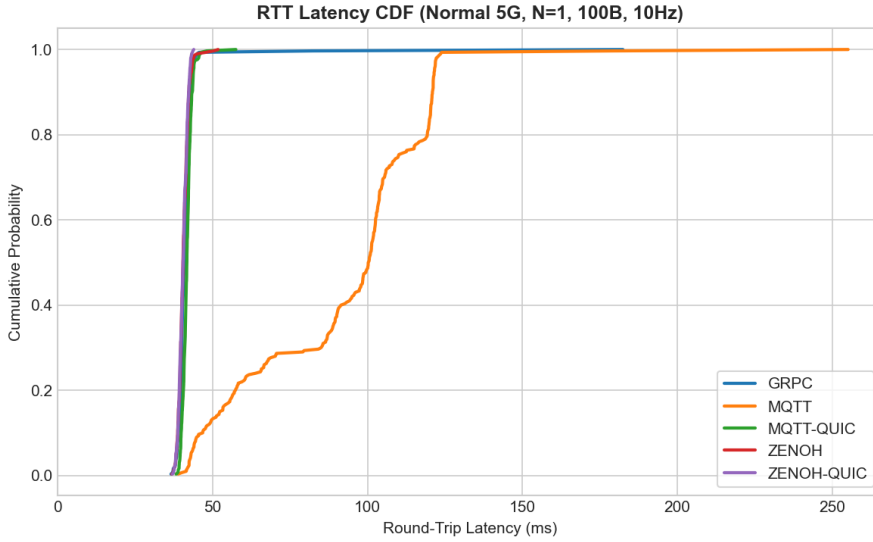


Figure 6: RTT latency CDF under normal 5G ($N = 1$, 100 B, 10 Hz). MQTT TCP is shifted right by ~ 45 ms. Zenoh, Zenoh-QUIC, and MQTT-QUIC all cluster at the 43 ms network floor.

4 What this means for CAMPUS

Table 4: Protocol comparison at $N = 10$, 2000 B payload, 10 Hz (Server-Run)

Protocol	Clean p50	Normal 5G p50	Degraded p50	Msg loss	Scales to $N = 20$?
Zenoh-QUIC	0.59 ms	40 ms	162 ms	<1%	Yes
Zenoh (TCP)	0.61 ms	40 ms	163 ms	<1%	Yes
MQTT-QUIC	3.56 ms	43 ms	213 ms	<1%	Yes
gRPC	2.91 ms	41 ms	163 ms	<1%	No (50% loss at $N = 20$)
MQTT (TCP)	45 ms	106 ms	34,946 ms	1–2%	No (queue collapse)

The server results confirm our protocol recommendations with higher precision:

1. **Zenoh-over-QUIC is the primary choice for the pub/sub layer.** It offers the lowest overhead (~ 0.5 ms baseline), zero message loss at $N = 10$, flat latency curves to $N = 20$, and robust performance under packet loss.
2. **MQTT-over-QUIC is viable if a central broker is required.** Swapping the transport to QUIC keeps latency bounded to ****213 ms**** under degraded conditions compared to the 35 seconds of MQTT TCP.
3. **gRPC is restricted to point-to-point data transfers.** The 50% message loss at $N = 20$ indicates an architectural limitation of HTTP/2 stream concurrency under high fan-out, disqualifying gRPC as a broadcast/group communication protocol.
4. **MQTT over TCP is disqualified.** A RTT of 95.9 seconds at $N = 20$ degraded represents total system failure.

5 Limitations and Next Steps

- **Physical 5G radio validation:** The next phase will run a subset of this matrix on a physical 5G testbed. We will start with a local 5G SA core using Open5GS + UERANSIM to validate our deployment workflow.
- **QUIC Client socket-close hardening:** We must address the client-side deadlock in NanoSDK's `nng_close`. A proposed solution is to reorder the connection lifecycle: sleep *before* connecting (to allow the broker's QUIC listener to bind fully) and connect right before active transmission, keeping the QUIC socket active and preventing teardown races.
- **ARM64 image rebuilds:** Before deploying to the Jetson Orin edge boards, we need to rebuild our protocol binaries (especially the MQTT-QUIC C binaries) for ARM64 using multi-arch Docker builds.